This File Is 1eb027a9844495e9a3c64bc0c7ea645058933a9b18cc98ff3f42a7b1a9142753

This file is a 32 bit Windows PE executable compiled on December 5, 2019. Its size is 40,960 bytes.

It runs as a GUI application, so no console window appears when it starts.

The executable loads at `0x400000` and has an entry point at `0x1000`.

It contains four main sections:

1. `.text` contains the program code.
2. `.rdata` contains read only data, including import information.
3. `.data` contains writable data and the malware configuration.
4. `.reloc` contains information used for address relocation.

It Does

The file appears to be a botnet client.

After execution, it connects to a Command and Control server and sends information about the infected computer. It then waits for commands from the server.

One of the main features of this sample is its built in Tor client. Instead of relying on a separate Tor installation, the executable contains its own Tor related code. This can hide the real location of the C2 server.

It also uses custom DNS servers instead of relying only on the Windows DNS configuration.

Configuration

The `.data` section starts with the marker `BEGINDATA` and contains the configuration in plain text.

The configuration contains:

Setting	Value
C2 domain	`bernieforweeed.com`
C2 IP	`94.158.247.29`
C2 port	`4001`
DNS 1	`5.132.191.104`
DNS 2	`92.163.33.248`
DNS 3	`206.189.120.27`
Process name	`a2guard.exe`
Persistence key	`start2`
Install location	`ALLUSERSPROFILE\Microsoft\win32app`

The domain and hardcoded IP provide two possible C2 addresses. If the domain becomes unavailable, the malware can use the IP address.

The DNS addresses are used to resolve the C2 domain. The configuration also contains a reference to `opennic.glue`, suggesting the malware can use the OpenNIC DNS network.

Installation and Persistence

The malware copies itself to the installation directory as:

`a2guard.exe`

It then creates a registry Run key named:

`start2`

This allows the malware to start automatically when Windows starts.

It also creates a mutex so that only one instance runs at a time.

The executable uses `asInvoker`, meaning it does not request administrator privileges. Therefore, it normally does not trigger a UAC prompt.

The following Windows APIs are used during this process:

1. `GetModuleFileNameA` to find its own file path.
2. `CopyFileA` to copy itself.
3. `CreateDirectoryA` to create directories.
4. `GetVolumeInformationA` to collect disk information.
5. `GetEnvironmentVariableA` to read environment variables.
6. `GetCommandLineW` to read the command line.
7. `GetUserNameW` to obtain the current username.

Communication With the C2 Server

The malware has two main communication methods.

Direct TCP

The first method uses normal TCP communication.

It uses Windows socket APIs such as:

`WSAStartup`

`socket`

`connect`

`send`

`recv`

The connection uses port `4001`.

It also uses `select`, `ioctlsocket`, and `setsockopt` for socket handling and connection settings.

This communication method is easier for network monitoring systems to detect.

Tor Communication

The second method uses the Tor network.

The executable contains its own Tor client code instead of depending on an external Tor installation.

It contains the addresses of several Tor directory authority servers, including:

`128.31.0.34`

`128.31.0.39`

`193.23.244.244`

`86.59.21.38`

`199.58.81.140`

`204.13.164.118`

`194.109.206.212`

`131.188.40.189`

`154.35.175.225`

`171.25.193.9`

The malware requests the Tor consensus information to find active relays.

It searches for relays with flags such as:

`Fast`

`Running`

`Valid`

`HSDir`

`Exit`

It also requests relay descriptors and parses information related to Tor hidden services.

The configuration contains a `TOR:` field without a value. This suggests that Tor communication may be controlled through configuration or commands received from the C2 server.

Finding the Public IP Address

The malware contacts two external services to find the infected machine's public IP address:

`api.ipify.org`

`ip4.seeip.org`

It uses a Firefox 66 user agent when making these requests.

Using two services provides a fallback if one service is unavailable.

The public IP address can then be sent to the C2 server as part of the victim information.

7. Cryptography

The sample uses several Windows cryptography and security APIs.

Windows CryptoAPI

It uses the Windows CryptoAPI with the:

`Microsoft Enhanced DSS and Diffie Hellman Cryptographic Provider`

APIs such as `CryptAcquireContextA`, `CryptImportKey`, and `CryptExportKey` are used for key handling and Diffie Hellman based key exchange.

Windows CNG

The malware also uses the CNG API from `bcrypt.dll`.

Important functions include:

`BCryptOpenAlgorithmProvider`

`BCryptEncrypt`

`BCryptImportKey`

`BCryptCreateHash`

`BCryptHashData`

`BCryptFinishHash`

These functions are used for encryption and hashing.

SSPI

The sample also uses SSPI from `secur32.dll`.

It uses:

`AcquireCredentialsHandleA`

`InitializeSecurityContextA`

`EncryptMessage`

`DecryptMessage`

The referenced provider is the Microsoft Unified Security Protocol Provider.

It also uses `GetUserNameExA` to collect username information in different formats.

Certificate and Encoding Functions

The malware uses:

`CryptDecodeObject`

`CryptStringToBinaryA`

These functions are useful for decoding and parsing certificate and encoded data used by its Tor related functionality.

Overall, the sample uses several layers of cryptographic protection for its communication.

System Information Collection

The malware collects information about the infected computer.

It uses:

`CreateToolhelp32Snapshot`

`Process32First`

`Process32Next`

These functions allow it to enumerate running processes.

This can be used to identify security software and other programs running on the system.

It also uses `IsWow64Process` to determine whether it is running under 32 bit emulation on a 64 bit Windows system.

`RtlGetVersion` from `ntdll.dll` is used to obtain the Windows version.

The malware also checks open windows using:

`EnumWindows`

`GetClassNameA`

`GetWindowTextA`

This can help identify virtual machine software, security products, and other applications.

`GetModuleFileNameExA` is used to obtain the full path of running processes.

Anti Analysis Techniques

One notable technique is the use of `LdrLoadDll` from `ntdll.dll` instead of the more common `LoadLibrary` function.

Some security tools monitor DLL loading by hooking common Windows APIs. Using a lower level function can help the malware avoid some basic monitoring.

The executable also uses the GUI subsystem, so there is no visible console window.

Imports such as:

`RegisterClassA`

`CreateWindowExA`

`ShowWindow`

suggest that it may create a hidden window for processing Windows messages.

The malware does not require administrator privileges and does not display a UAC prompt.

It also uses `CreateThread` to create multiple threads and uses `InterlockedExchange` and `InterlockedCompareExchange` for thread safe operations.

Possible Malware Family

Based on the available evidence, this sample appears to be related to the **Andromeda, also known as Gamarue, botnet family**, or a closely related variant.

The main indicators are:

1. The use of `a2guard.exe` as the process name.
2. The registry based persistence.
3. The Tor integration.
4. The custom C2 communication.
5. The layered cryptographic implementation.
6. The December 2019 compilation date.

The process name and overall structure are consistent with known Andromeda related samples.

This identification should be treated as a family assessment based on the observed characteristics rather than as definitive attribution without additional indicators such as hashes, matching code, configuration patterns, or known threat intelligence.

Execution Flow

The overall execution flow can be summarized as follows:

1. The malware starts.
2. It creates a mutex to check whether another instance is already running.
3. It copies itself to `ProgramData\Microsoft\win32app\a2guard.exe`.
4. It creates the `start2` registry Run key for persistence.
5. It collects information about the operating system and username.
6. It checks running processes and open windows.
7. It collects hardware and disk related information.
8. It finds the public IP address.
9. It resolves the C2 domain using the configured DNS servers.
10. It connects to the C2 server on port `4001`.
11. It performs the required key exchange and encryption setup.
12. It registers the infected machine with the C2 server.
13. It waits for commands.
14. If Tor communication is enabled, the C2 communication can instead be routed through the embedded Tor functionality.

Final Assessment

This sample is a persistent Windows botnet client with several capabilities designed to maintain communication with its C2 infrastructure.

The main characteristics are:

1. Persistent execution through a Windows Run key.
2. Background execution without a visible console.
3. Collection of system and process information.

4. Direct TCP based C2 communication.
5. Embedded Tor functionality.
6. Custom DNS resolution.
7. Multiple cryptographic mechanisms.
8. Basic anti analysis techniques.
9. Ability to identify security software and virtual machine environments.

The combination of persistence, system information collection, encrypted C2 communication, Tor support, and anti analysis techniques makes this a sophisticated botnet sample for malware analysis.